Assignment 1 Solutions

Process Abstractions, Scheduling & Performance Analysis

Advay Sangrulkar

May 2026

Part 1: Process Abstraction & the POSIX API

1.1 State Machines Under Pressure

(a) Timer Interrupt to Process Resume

When a hardware timer fires while a process is executing, the following exact sequence occurs:

1. **Hardware saves minimal state & vectors to kernel.** The CPU pushes the program counter (PC) and processor status word (PSW) onto the kernel stack and jumps to the timer interrupt handler via the interrupt descriptor table (IDT).
2. **Context save into PCB.** The interrupt handler saves the full CPU context (all general-purpose registers, stack pointer, segment registers) of the currently-running process into that process's *Process Control Block* (PCB). The PCB's `state` field is updated from `RUNNING` to `READY`.
3. **Run-queue update.** The kernel places the just-preempted process's PCB back onto the *run queue* (ready queue), a data structure (commonly a priority queue, multi-level feedback queue array) ordered by scheduling policy.
4. **Scheduler invocation.** The timer handler calls `schedule()`, which reads the run queue to select the highest-priority READY process according to the scheduling algorithm.
5. **Context restore from new PCB.** The kernel loads the CPU context from the selected process's PCB (registers, SP, PC). The selected PCB's `state` is set to `RUNNING`.
6. **Return from interrupt.** An `iret` (or equivalent) instruction restores the saved PC and PSW and switches privilege level, resuming the new process in user mode at exactly the instruction it was last at.

Data structures read/written:

- **PCB** (read & written): context saved for old process; context restored for new process; state fields updated.
- **Run queue** (read & written): old process inserted; new process removed.

- **IDT** (read only): used to dispatch the timer interrupt to the correct handler.
- **Kernel stack** (written then read): hardware pushes PC/PSW; handler saves/restores registers.

(b) Incorrect Transition Sequence Analysis

The claimed sequence is: **Running → Terminated → Blocked → Ready → Running**

(i) Invalid transitions:

1. **Running → Terminated:** A process calling `read()` on an empty network socket does *not* terminate. `read()` is a blocking I/O call; the process has not exited. Termination only occurs via `exit()`, `_exit()`, or a fatal signal. This transition is impossible here.
2. **Terminated → Blocked:** Once a process enters the `Terminated` (zombie) state, it *cannot* transition to any other state. Terminated is a terminal state in the kernel state machine. A dead process has no execution context and cannot issue system calls or perform I/O.
3. **Terminated → Blocked (and onward):** Every subsequent transition from `Terminated` is invalid for the same reason.

(ii) Correct transition sequence:

Step	Transition	Triggering Event	Originator
1	Running → Blocked	Process calls `read()`; no data available; kernel suspends the process	Process (system call)
2	Blocked → Ready	Network data arrives; NIC fires hardware interrupt; kernel moves process to run queue	Hardware interrupt
3	Ready → Running	Scheduler selects this process and dispatches it	OS Scheduler

(c) Zombie Processes

A **zombie process** retains its entry in the process table for one reason: the kernel must preserve its *exit status* until the parent retrieves it. Even though the process has stopped executing and all of its resources (memory, file descriptors, open handles) have been released, the PCB entry (specifically: PID, exit status, CPU usage accounting) must persist so that the parent's eventual `wait()` call can collect that information. If the kernel removed the PCB immediately upon process exit, the exit status would be lost permanently.

System call to remove a zombie: `wait()` or `waitpid()` (POSIX). When the parent calls either, the kernel copies the exit status to the parent's address space and then removes the PCB entry, freeing the PID.

If the parent exits first: The orphaned zombie (and any still-running children) are *reparented* to the `init` process (PID 1, or `systemd` on modern Linux). `init` periodically calls `wait()` to reap all adopted children, preventing PID table exhaustion. Without this

mechanism, zombies whose parents have died could accumulate indefinitely, leaking PID table slots.

1.2 Deep fork() Analysis

(a) Process Proliferation Formula

Unconstrained loop formula: If `fork()` is called inside a loop executing n times with no `exit()` inside the loop, every existing process duplicates on each iteration. After iteration k, the total number of processes doubles. Starting with 1 process:

$$\text{Total processes after } n \text{ iterations} = 2^n$$

Reasoning: Before iteration 1, there is 1 process. After iteration 1, $1 \times 2 = 2$. After iteration 2, $2 \times 2 = 4$. By induction, after n iterations: 2^n processes.

Verification against the given program:

The program performs **Fork A** at the top level, then **Fork B** only in the child of A. This is *not* an unconstrained loop:

- Fork A creates 1 child. Total: 2 processes (A-parent, A-child).
- Fork B is called only by A-child, creating 1 more child. Total: 3 processes (Process A, Process B, Process C).
- Both fork calls execute *at most once* per process, and Fork B is guarded by the `if (rc1 == 0)` branch.

The program creates **3 processes total**, not $2^2 = 4$, because the second fork is conditional on being the child of the first fork. In a true unconstrained loop, all existing processes (including the original parent) execute each fork, producing exponential growth.

(b) Exact Output

Initial values: global `g = 10`, stack `x = 5`.

After **Fork A**: Process A (parent) has `g=10, x=5`; Process B (child of A) has `g=10, x=5` (copy-on-write copy).

After **Fork B** (inside Process B): Process B retains `g=10, x=5`; Process C (child of B) has `g=10, x=5`.

Process C executes: `g += 100; x += 100;` ⇒ `g=110, x=105`.

Exact output:

```
C: g=110 x=105
B: g=10 x=5
A: g=10 x=5
```

Justification:

- **Process C** prints `g=110, x=105` because it added 100 to both variables.

- **Process B** prints `g=10, x=5` because it calls `wait(NULL)` for C first, then prints. B's own copy of `g` and `x` were *never modified*. The modifications in C are invisible to B.
- **Process A** prints `g=10, x=5` because it calls `wait(NULL)` for B first, then prints. A's copy of `g` and `x` are also unmodified.

Why `g` does not propagate between processes: `fork()` creates a *copy* of the parent's address space for the child using **copy-on-write (COW)**. Although both parent and child initially reference the same physical pages, the moment either process writes to a page (e.g., modifying `g`), the kernel creates a private copy of that page for the writing process. From that point on, each process has an entirely independent copy of `g` in its own virtual address space. There is no shared memory between processes unless explicitly established via `mmap(MAP_SHARED)`, `shmget()`, or similar IPC mechanisms.

(c) Ordering Guarantees

The **happens-before** constraints are:

- **C prints before B prints**: Process B calls `wait(NULL)`, which blocks until C terminates. C must print before it terminates via `exit(0)`, which happens before B's `wait()` returns, which is before B prints.
- **B prints before A prints**: Process A calls `wait(NULL)`, which waits for B to terminate. B prints before calling `exit(0)`, which happens before A's `wait()` returns.
- By transitivity: **C prints before A prints**.

Therefore: C-print → B-print → A-print (strict total order).

Impossible orderings (any ordering that violates the above):

1. `B: ...` before `C: ...` — impossible because B waits for C.
2. `A: ...` before `B: ...` — impossible because A waits for B.
3. `A: ...` before `C: ...` — impossible by transitivity.
4. `A: ...` first in any position — impossible.
5. `B: ...` first in any position — impossible.

The **only valid ordering** is: `C` prints, then `B` prints, then `A` prints.

(d) Modified Scenario

(i) Heap sharing after fork():

No, the child and grandchild do **not** share the same heap object after `fork()`. `fork()` duplicates the entire virtual address space using **copy-on-write (COW)**. The child receives a separate virtual address space where the pointer `x` holds the same virtual address, but this virtual address maps to a *private physical page* once either process writes to it. The OS mechanism responsible is the **virtual memory subsystem**: both processes initially share the physical page (as read-only COW), but on the first write, the MMU triggers a page fault, and the kernel allocates a new physical page for the writing

process, copying the original content. After this, the two processes's pointer `x` point to the same virtual address but different physical memory.

(ii) Class of bug when parent frees before child finishes:

This is a **use-after-free (UAF)** bug. The parent calls `free(x)`, which returns that memory to the allocator. If the child (which has a COW copy of the same virtual page) then reads or writes through its copy of the pointer, it accesses memory that may have been reallocated for another purpose in the parent or may produce undefined behavior in the child's own address space if the page was not yet copied.

Detection methods:

- **Valgrind** (`memcheck`): instruments all memory accesses and detects reads/writes after `free()`.
- **AddressSanitizer (ASan)**: compile with `-fsanitize=address`; poisons freed memory and reports any subsequent access with a full stack trace at runtime.

(iii) State of Process C after B exits without wait():

Immediately after Process B calls `exit(0)` without waiting for C, Process C becomes an **orphan process**. It continues executing normally — its state is `RUNNING` or `READY` (whichever it was before B exited).

OS mechanism: The kernel's `exit()` path contains logic to *reparent* all children of the exiting process. C's `parent_pid` field in its PCB is updated to point to **PID 1** (`init` / `systemd`).

Broader system resource risk: If processes repeatedly exit without reaping their children, and `init` is slow or fails to reap, **zombie processes** can accumulate. Each zombie holds a PID and a PCB entry. Since the PID namespace is finite (typically 32,768 PIDs on Linux by default), exhausting it prevents *any* new process from being created system-wide. This is a **PID table exhaustion** (resource leak) vulnerability.

Part 2: Scheduling Algorithms & Trade-offs

2.1 Multi-Process Workload Analysis

Workload:

Process	Arrival (ms)	Burst (ms)	Priority	Remaining at arrival
P1	0	10	3	10
P2	2	4	1	4
P3	4	6	2	6
P4	6	2	4	2
P5	8	8	2	8

(a) SRTF Preemption Events

Under SRTF (Shortest Remaining Time First), we track remaining burst times at each potential decision tick.

(i) Preemption ticks:

1. **t = 0:** P1 arrives and starts (only process). No preemption, P1 begins running.
2. **t = 2: Preemption occurs.** P1 has been running for 2 ms; P1 remaining = $10 - 2 = 8$ ms. P2 arrives with remaining = 4 ms. Since $4 < 8$, P2 preempts P1. *Preempted: P1; takes over: P2.*
3. **t = 4: Preemption occurs.** P2 has run for 2 ms; P2 remaining = $4 - 2 = 2$ ms. P3 arrives with remaining = 6 ms. Since $2 < 6$, P2 is *not* preempted (P2 still has shorter remaining time). P4 is not here yet. No preemption at t=4.
4. **t = 6: P2 finishes at t = 6.** At t=6, P2 completes (ran from t=2 to t=6, burst=4). P4 arrives with remaining=2. Currently available: P1 (rem=8), P3 (rem=6), P4 (rem=2). Shortest is P4 (2 ms). *P4 starts. Not a preemption event per se, but a scheduling decision point.*
5. **t = 8: P4 finishes.** P4 ran from t=6 to t=8. P5 arrives at t=8 with remaining=8. Available: P1 (rem=8), P3 (rem=6), P5 (rem=8). Shortest is P3 (6). *P3 starts.*
6. **t = 14: P3 finishes.** Available: P1 (rem=8), P5 (rem=8). Tie broken by arrival order: P1 starts.
7. **t = 22: P1 finishes.** P5 runs to completion at t=30.

Summary of preemption events (where a running process is displaced by a newly arriving one):

Tick	Preempted	Takes Over	Reason
$t = 2$	P1 (rem=8)	P2 (rem=4)	P2 arrives with shorter remaining time than P1

At $t = 4$: P3 arrives with rem=6; current runner P2 has rem=2. No preemption ($2 < 6$).
At $t = 6$: P2 finishes naturally; P4 (rem=2) selected from queue. Scheduling decision, not preemption.
At $t = 8$: P4 finishes; P3 (rem=6) selected. Scheduling decision, not preemption.

(ii) Can P1 be the last process to finish under SRTF?

Yes, and it does in this workload (P1 finishes at $t = 22$, before P5 finishes at $t = 30$). Actually, let us recheck: P5 finishes at $t = 30$, P1 finishes at $t = 22$, so P5 is last.

More generally: P1 *can* be the last to finish under SRTF if all other processes have burst times $\geq$ P1's remaining time whenever they arrive i.e., no other process ever preempts P1, and P1 has a large initial burst. The structural condition is: **all other processes have burst times strictly greater than P1's remaining burst at their arrival time**, so P1 is never preempted and runs to completion before others with larger bursts. In this workload, P1 is *not* the last to finish because P5 (burst=8) arrives late and has equal remaining time to P1's remaining at $t = 14$, but P3 and P4 are shorter and delay P1 substantially. P1 finishes at $t = 22$, P5 at $t = 30$.

(b) Non-Preemptive SJF vs. FCFS: Structural Argument

Under **FCFS**, the order of execution is determined by arrival time: P1, P2, P3, P4, P5 (since P1 arrives first and no preemption occurs, it runs to completion). P1's burst of 10

ms forces all subsequent processes to wait.

Under **non-preemptive SJF**, when P1 completes (t=10), the ready queue contains P2 (burst=4), P3 (burst=6), and P4 (burst=2). SJF will select P4 (burst=2) first, then P2 (burst=4), then P3 (burst=6), then P5 (burst=8) when it arrives.

The key structural mechanism: **SJF re-orders the processing of jobs that arrive while P1 is running**. In FCFS, these jobs execute in arrival order (P2, P3, P4), meaning P3 and P4 wait behind P2. In SJF, shorter jobs (P4, P2) execute before P3, reducing the total waiting accumulated by P2 and P4. The process responsible for the difference is **P4** (burst=2): under FCFS it waits behind both P2 and P3 (which have larger bursts), but under SJF it executes immediately after P1 finishes. This reduces P4's wait time substantially, pulling down the average. P2 also benefits by executing before P3 in SJF regardless of arrival. Hence SJF $\leq$ FCFS for average wait time on this workload.

(c) Round Robin ($q = 3$ ms)

Simulation trace (queue managed as FIFO; new arrivals added at end of each tick when a quantum expires or process finishes):

Time interval	Process	Notes
0–3	P1	P1 starts. At t=2, P2 arrives (added to queue). At t=3, quantum ends; P1
3–6	P2	P2 runs. At t=4, P3 arrives. At t=6, P2 finishes (burst=4, only 3 ms of qu
		wait: P2 ran from 3–6 using all 4ms? Let us recount: P2 burst=4; gets qua
6–9	P1	P1 runs 3ms (rem: 7→4). At t=8, P5 arrives. Queue: [P3, P4, P2(rem=1),
9–12	P3	P3 runs 3ms (rem: 6→3). Queue: [P4, P2(rem=1), P5, P1, P3]
12–14	P4	P4 runs 2ms (burst=2, finishes at t=14). Queue: [P2(rem=1), P5, P1, P3]
14–15	P2	P2 runs 1ms remaining, finishes at t=15. Queue: [P5, P1, P3]
15–18	P5	P5 runs 3ms (rem: 8→5). Queue: [P1, P3, P5]
18–21	P1	P1 runs 3ms (rem: 4→1). Queue: [P3, P5, P1]
21–24	P3	P3 runs 3ms (rem: 3→0), finishes at t=24. Queue: [P5, P1]
24–27	P5	P5 runs 3ms (rem: 5→2). Queue: [P1, P5]
27–28	P1	P1 runs 1ms remaining, finishes at t=28. Queue: [P5]
28–30	P5	P5 runs 2ms remaining, finishes at t=30.

Finish times and wait/turnaround:

Using the formula: Wait Time = Finish − Arrival − Burst.

Process	Arrival	Burst	Finish	Turnaround T_i	Wait x_i
P1	0	10	28	28	18
P2	2	4	15	13	9
P3	4	6	24	20	14
P4	6	2	14	8	6
P5	8	8	30	22	14

$$\text{Average Wait} = \frac{18+9+14+6+14}{5} = \frac{61}{5} = \boxed{12.2 \text{ ms}}$$

$$\text{Average Turnaround} = \frac{28+13+20+8+22}{5} = \frac{91}{5} = \boxed{18.2 \text{ ms}}$$

Why is RR's average turnaround higher than SRTF's?

SRTF always runs the process with the shortest remaining time, completing short jobs quickly and minimising the time any process spends waiting. Round Robin distributes the CPU in fixed-size quanta regardless of how much time each process still needs, causing long and short jobs to interleave. This means short jobs that could have finished early (e.g., P4 with burst=2) still must wait for other processes' quanta before getting the CPU again. Every context switch adds overhead and delays the completion of jobs, inflating turnaround times across the board. Fairness in RR is achieved by sharing the CPU time, but this sharing comes at the cost of each individual job completing later than it would under an optimal preemptive policy like SRTF.

(d) Non-Preemptive SJF

Under non-preemptive SJF, once a process starts it runs to completion. The scheduler picks the shortest burst among all *ready* processes at each decision point.

Execution trace:

- **t=0:** Only P1 ready. P1 starts (burst=10). Runs until t=10.
- **t=10:** P1 finishes. Ready: P2(4), P3(6), P4(2). Shortest: P4(2). P4 starts.
- **t=12:** P4 finishes. Ready: P2(4), P3(6), P5(8) [P5 arrived at t=8]. Shortest: P2(4). P2 starts.
- **t=16:** P2 finishes. Ready: P3(6), P5(8). Shortest: P3(6). P3 starts.
- **t=22:** P3 finishes. Only P5 ready (burst=8). P5 starts.
- **t=30:** P5 finishes.

Process	Arrival	Burst	Start	Finish	Wait x_i = Start − Arrival
P1	0	10	0	10	0
P4	6	2	10	12	4
P2	2	4	12	16	10
P3	4	6	16	22	12
P5	8	8	22	30	14

Turnaround T_i = Finish − Arrival:

Process	Turnaround
P1	10
P4	6
P2	14
P3	18
P5	22

$$\text{Average Wait} = \frac{0+4+10+12+14}{5} = \frac{40}{5} = \boxed{8.0 \text{ ms}}$$

$$\text{Average Turnaround} = \frac{10+6+14+18+22}{5} = \frac{70}{5} = \boxed{14.0 \text{ ms}}$$

2.2 Theoretical Bounds

(a) SJF Optimality for $n = 3$ Simultaneous Jobs

Claim: Among all non-preemptive orderings of $n = 3$ jobs with burst times $b_1 \leq b_2 \leq b_3$ arriving at $T = 0$, the ordering (b_1, b_2, b_3) minimises average wait time.

Proof:

Let the three jobs run in some order. Without loss of generality, denote their burst times in execution order as $(b_{\sigma(1)}, b_{\sigma(2)}, b_{\sigma(3)})$ for a permutation σ.

The wait times are:

$$w_1 = 0, \quad w_2 = b_{\sigma(1)}, \quad w_3 = b_{\sigma(1)} + b_{\sigma(2)}$$

The average wait time is:

$$\bar{w}(\sigma) = \frac{0 + b_{\sigma(1)} + b_{\sigma(1)} + b_{\sigma(2)}}{3} = \frac{2b_{\sigma(1)} + b_{\sigma(2)}}{3}$$

To minimise $\bar{w}$, we minimise $2b_{\sigma(1)} + b_{\sigma(2)}$. Since $b_1 \leq b_2 \leq b_3$:

- $b_{\sigma(1)}$ has coefficient 2, so it should be as small as possible: $b_{\sigma(1)} = b_1$.
- Given $b_{\sigma(1)} = b_1$, we minimise $b_{\sigma(2)}$ from the remaining $\{b_2, b_3\}$: $b_{\sigma(2)} = b_2$.

Therefore the optimal order is (b_1, b_2, b_3) — i.e., SJF — yielding:

$$\bar{w}_{\text{SJF}} = \frac{2b_1 + b_2}{3}$$

Any other ordering produces a larger or equal value. For example, order (b_1, b_3, b_2):

$$\bar{w} = \frac{2b_1 + b_3}{3} \geq \frac{2b_1 + b_2}{3} \quad \text{since } b_3 \geq b_2. \checkmark$$

Order (b_2, b_1, b_3):

$$\bar{w} = \frac{2b_2 + b_1}{3} \geq \frac{2b_1 + b_2}{3} \iff b_2 \geq b_1. \checkmark$$

All other orderings are dominated by the SJF ordering. ■

(b) SRTF vs. Round Robin: Closed-form Response Times

Setup: 1 long job (L ms) and k short jobs (each s ms, $s \ll L$), all arrive simultaneously at $t = 0$.

(i) Response times:

Under SRTF:
All short jobs have remaining time s; the long job has $L \gg s$. SRTF will run all short jobs to completion before touching the long job.

- Short job i (for $i = 1, \ldots, k$): starts at time $(i-1)s$, finishes at is. Response time $= (i-1)s$.

- Long job: starts at ks, finishes at $ks + L$. Response time = ks.

$$\text{Avg Response Time}_{\text{SRTF}} = \frac{\sum_{i=1}^{k}(i-1)s + ks}{k+1} = \frac{s \cdot \frac{k(k-1)}{2} + ks}{k+1} = \frac{s \cdot \frac{k(k+1)}{2}}{k+1} = \frac{ks}{2}$$

Under Round Robin (quantum $q < s$):
All $k+1$ jobs receive the CPU in rotation. With quantum q, in each full round of $k+1$ quanta, each job gets q ms of CPU. Short jobs each need $\lceil s/q \rceil \approx s/q$ rounds. The long job needs L/q rounds.

Short job i's first quantum is served at time $(i-1)q$. Assuming all $k+1$ processes are in the queue, the response time (time to first scheduling) for short job i is $(i-1)q$. For the long job: response time = kq.

Average response time (time to first CPU):

$$\text{Avg Response Time}_{\text{RR}} = \frac{\sum_{i=1}^{k}(i-1)q + kq}{k+1} = \frac{q \cdot \frac{k(k-1)}{2} + kq}{k+1} = \frac{kq}{2}$$

(ii) Condition for RR to outperform SRTF:

$$\text{Avg RT}_{\text{RR}} < \text{Avg RT}_{\text{SRTF}} \iff \frac{kq}{2} < \frac{ks}{2} \iff q < s$$

This is always true by our assumption ($q < s$). Therefore, for this symmetric workload with equal-sized short jobs, **RR always has lower average response time than SRTF when $q < s$.**

Intuitive interpretation: SRTF runs short jobs sequentially — the k-th short job must wait for all $k-1$ others to finish. RR interleaves all jobs so each gets a turn quickly: the k-th job first runs at time $(k-1)q$ rather than $(k-1)s$. Since $q < s$, the initial wait under RR is much smaller. The cost of RR is that short jobs take longer to *complete* (they are interleaved with the long job and other shorts), but they *start* sooner. When k is large and $s \gg q$, RR's advantage in response time is substantial.

Part 3: Quantitative Performance & Advanced Metrics

3.1 Metric Derivation

Given trace (all processes arrive at $T = 0$):

Process	Start (ms)	Finish (ms)	Burst = Finish − Start
P1	0	12	12
P2	12	15	3
P3	15	22	7
P4	22	24	2
P5	24	30	6

(a) Wait and Turnaround Times

Since all processes arrive at $T = 0$:

$$x_i = \text{Start}_i - \text{Arrival}_i = \text{Start}_i$$

$$T_i = \text{Finish}_i - \text{Arrival}_i = \text{Finish}_i$$

Process	Wait Time x_i (ms)	Turnaround T_i (ms)
P1	0	12
P2	12	15
P3	15	22
P4	22	24
P5	24	30

$$\text{Average Wait} = \frac{0 + 12 + 15 + 22 + 24}{5} = \frac{73}{5} = 14.6 \text{ ms}$$

$$\text{Average Turnaround} = \frac{12 + 15 + 22 + 24 + 30}{5} = \frac{103}{5} = 20.6 \text{ ms}$$

(b) Makespan

$$\text{Makespan} = \max_i(f_i) - \min_i(a_i) = 30 - 0 = \boxed{30 \text{ ms}}$$

(c) Jain's Fairness Index

Wait times: $x_1 = 0,\ x_2 = 12,\ x_3 = 15,\ x_4 = 22,\ x_5 = 24.$

$$\sum_{i=1}^{5} x_i = 0 + 12 + 15 + 22 + 24 = 73$$

$$\left(\sum_{i=1}^{5} x_i\right)^2 = 73^2 = 5329$$

$$\sum_{i=1}^{5} x_i^2 = 0^2 + 12^2 + 15^2 + 22^2 + 24^2 = 0 + 144 + 225 + 484 + 576 = 1429$$

$$n \cdot \sum_{i=1}^{5} x_i^2 = 5 \times 1429 = 7145$$

$$J_{\text{FCFS}} = \frac{5329}{7145} \approx \boxed{0.746}$$

(d) Comparison: J_{SJF} vs J_{FCFS}

The burst sizes, derived from the FCFS trace, are: P1=12, P2=3, P3=7, P4=2, P5=6. Under SJF (all arrive at $T = 0$), execution order by ascending burst is: P4(2), P2(3), P5(6), P3(7), P1(12).

Wait times under SJF:

$$x_{\mathrm{P4}} = 0,\ x_{\mathrm{P2}} = 2,\ x_{\mathrm{P5}} = 5,\ x_{\mathrm{P3}} = 11,\ x_{\mathrm{P1}} = 18$$

In FCFS, P1 (the longest job) runs first and waits 0 ms, while P2–P5 wait increasingly long. This produces a highly skewed distribution: one process waits 0 ms but subsequent ones each wait much more, yielding high variance. In SJF, the longest-waiting process is P1 (wait=18), but it has one of the largest bursts. Shorter jobs wait less. The distribution is more "ordered" and less skewed in terms of variance relative to the mean. However, the mean wait under SJF is lower than FCFS.

Computing J_{SJF} quickly: $\sum x_i = 0+2+5+11+18 = 36$; $\sum x_i^2 = 0+4+25+121+324 = 474$; $J_{\mathrm{SJF}} = 36^2/(5 \times 474) = 1296/2370 \approx 0.547$.

$J_{\mathrm{SJF}} \approx 0.547 < J_{\mathrm{FCFS}} \approx 0.746$: $\boldsymbol{J_{\mathrm{SJF}}}$ **is less than** $\boldsymbol{J_{\mathrm{FCFS}}}$.

Justification: Under FCFS, P1 (the largest job) runs first and "pays" 0 wait time, while all smaller jobs accumulate increasing wait times. The distribution is monotonically increasing and bunched together (differences driven by burst sizes), but the first job being the longest means subsequent jobs experience a large initial penalty. Under SJF, the large job P1 is deferred until last, causing it to accumulate the maximum wait time (18 ms). This creates a wider spread of wait times (0, 2, 5, 11, 18) compared to FCFS (0, 12, 15, 22, 24 but lower mean). The higher variance relative to mean under SJF actually *reduces* the Jain index compared to FCFS's more "bunched" distribution: FCFS has a higher mean but the ratios $(\sum x_i)^2/(n\sum x_i^2)$ work out favourably for FCFS here, as verified numerically.

3.2 Interpreting Real-World Measurements

Scheduler	Mean Wait (ms)	P90 Wait (ms)	P99 Wait (ms)	Makespan (ms)
Alpha	8	12	18	950
Beta	6	280	620	900
Gamma	11	14	19	870
Delta	9	11	16	1100

(a) Algorithm Family for Beta

Most likely family: Priority Scheduling (or Shortest Job First / Shortest Remaining Time First with starvation).

Beta's low mean (6 ms) with extreme P99 (620 ms) is the hallmark of **starvation-prone algorithms** that aggressively favour short or high-priority jobs. When short or high-priority jobs dominate the workload, most jobs complete quickly (low mean), but any long or low-priority job can be indefinitely delayed. These tail cases show up at P90/P99.

Minimal concrete example reproducing this pattern:

Process	Arrival	Burst	Priority
A	0	1	High
B	0	1	High
C	0	60	Low

Under strict priority scheduling: A and B run first (wait = 0 and 1 ms), C runs last (wait = 2 ms). If instead of 3 processes there are thousands of high-priority short jobs arriving continuously, C (low priority) can wait hundreds or thousands of ms while the mean remains low because most jobs are short and high-priority. Mean $\approx$ 1 ms; P99 (C's wait) $\gg$ mean.

(b) Throughput vs. Tail Latency: Scheduler Gamma

A scheduler can minimise Makespan while producing higher average wait times by **prioritising total CPU utilisation and job completion rate over individual fairness**. Gamma's lower Makespan (870 ms) means it completes the entire workload of 10,000 jobs faster than Alpha or Beta as a batch. This is achieved by strategies such as scheduling jobs that enable maximum CPU utilisation (e.g., batching jobs that share cache lines, minimising idle time, or running the longest jobs first to ensure the CPU is never idle waiting for short jobs). However, maximising throughput often means small jobs must wait behind large ones in certain phases, or that context switches are minimised at the expense of some processes sitting in the queue longer, raising the mean wait.

Is Makespan a useful metric for interactive workloads? No. Interactive workloads care about *per-request latency* which is how long each individual user request takes. A low Makespan only tells us that the batch of all jobs finishes quickly in aggregate; it says nothing about whether any individual user experienced a 50 ms delay (which would be unacceptable for a chat application, for example). Makespan is the right metric for batch/throughput systems where total wall-clock time is the primary concern, but it is misleading for interactive systems where tail latency (P90, P99) and mean response time per request drive user experience.

(c) Deployment Decision

System (i): Batch Genomics Pipeline

Best choice: Scheduler Gamma.

Justification:

- **Makespan = 870 ms** (lowest among all schedulers): the genomics pipeline runs overnight and cares only about finishing all jobs as quickly as possible. Gamma completes the entire batch fastest.
- **P90 Wait = 14 ms**: acceptable for batch work — individual job latency is not a concern as long as the overall batch completes quickly.
- Higher mean wait (11 ms) is irrelevant for a batch system where throughput is paramount.

System (ii): Real-Time Chat Application

Best choice: Scheduler Delta.

Justification:

- **P90 Wait = 11 ms and P99 Wait = 16 ms** (lowest P99 among all schedulers): the critical requirement is that no message takes more than 50 ms to process. Delta's P99 of 16 ms provides a comfortable margin — 99% of messages are processed in ≤ 16 ms, well within the 50 ms threshold.
- **Mean Wait = 9 ms**: slightly higher than Alpha (8 ms) and Beta (6 ms), but the excellent tail behaviour makes Delta far superior for interactive use.
- Beta's catastrophic P99 = 620 ms disqualifies it entirely (messages would visibly lag). Alpha's P99 = 18 ms is acceptable, but Delta's P99 = 16 ms and P90 = 11 ms make it the safer choice with lower tail risk.
- The higher Makespan of Delta (1100 ms) is irrelevant for a real-time system where overall batch throughput does not matter.